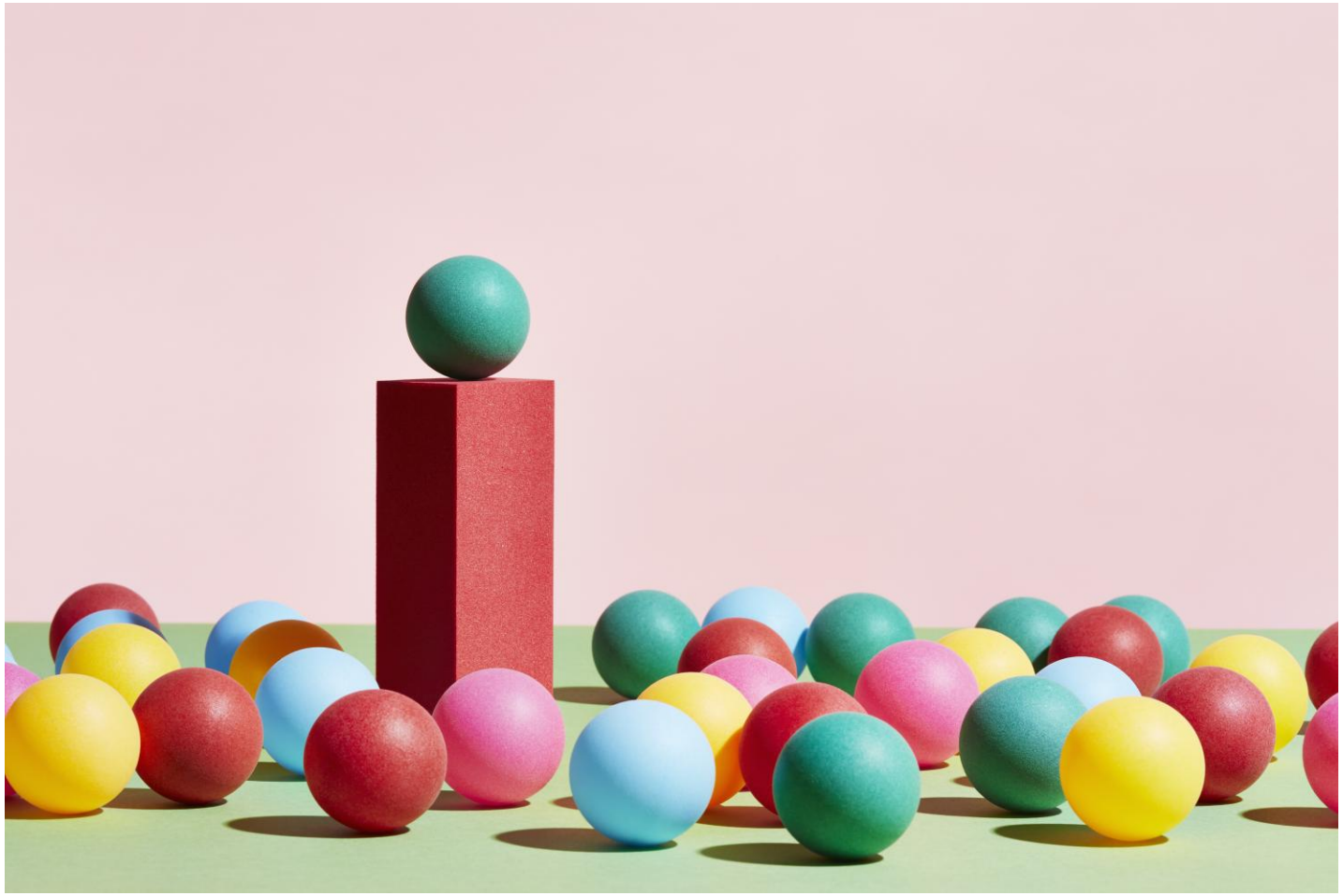
Informatics 3A

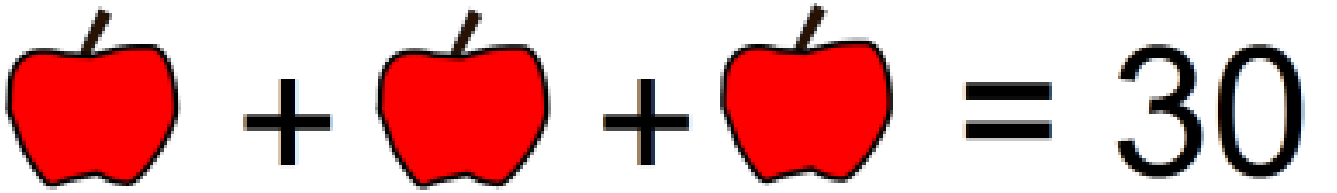
Pattern-Based Design

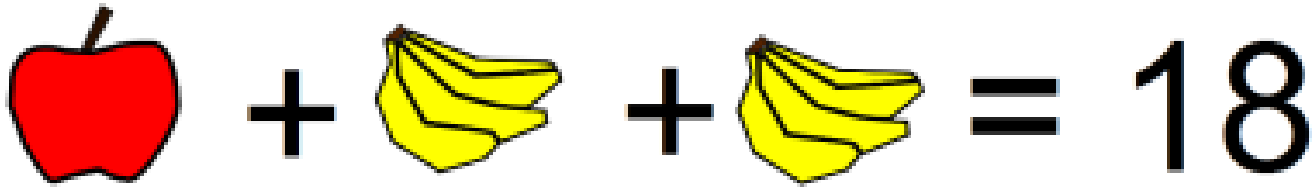
UNIVERSITY OF JOHANNESBURG

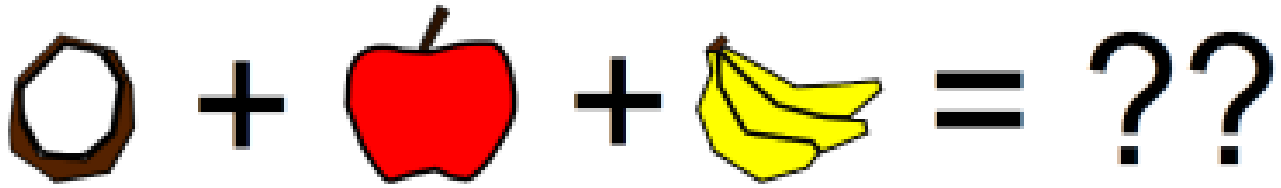

Your past mistakes
are meant to guide you,
not define you.

Lets play a game…

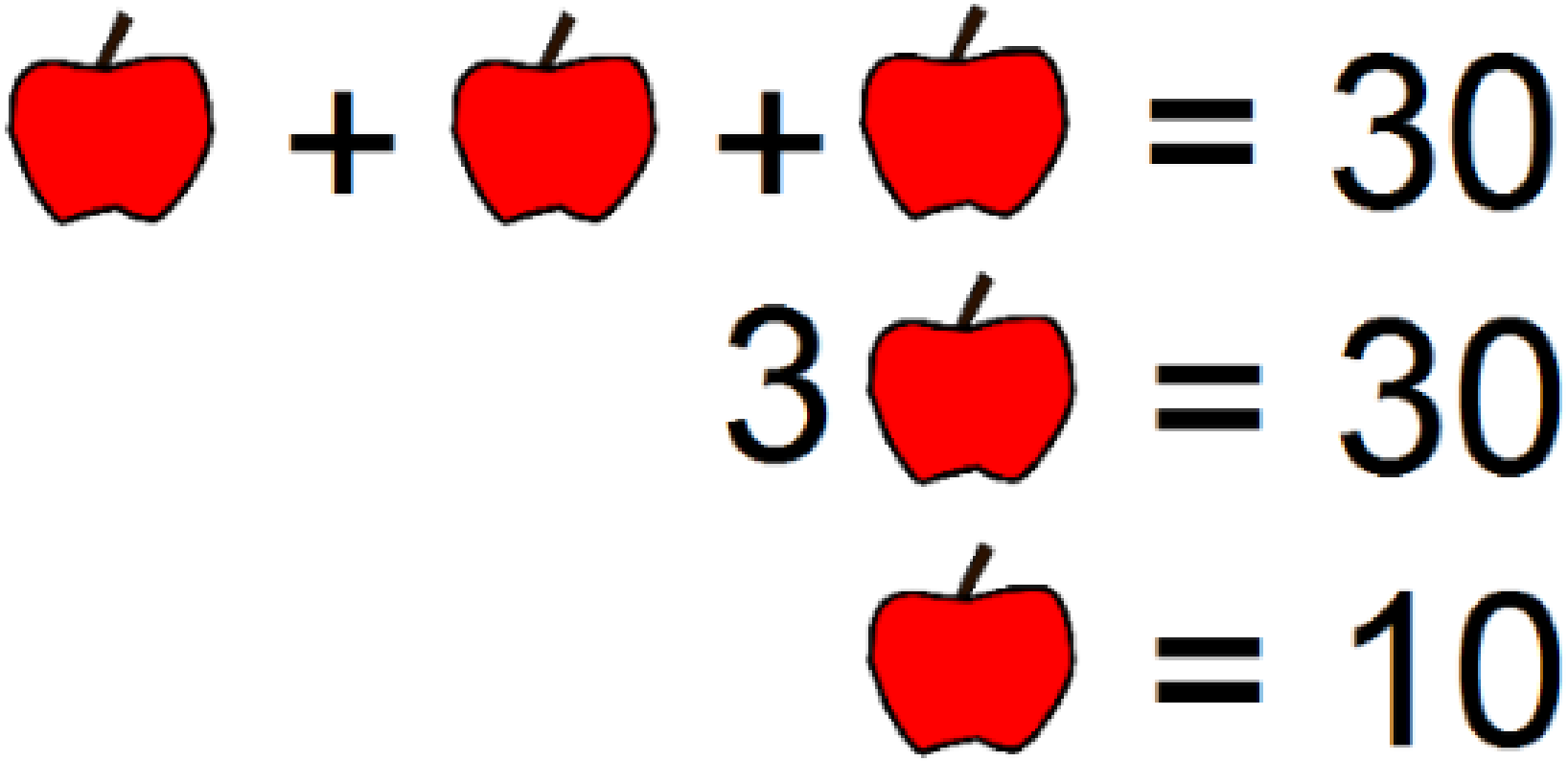




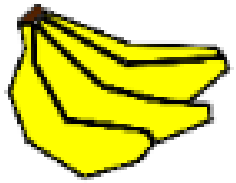
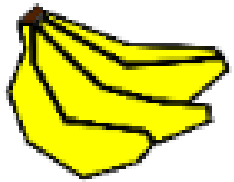


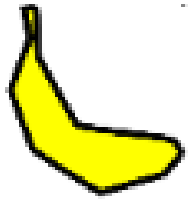


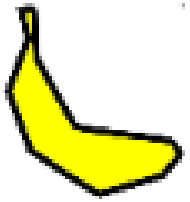
Mind Your Decisions

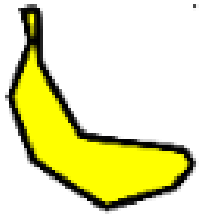

+ + = 30
3 = 30
= 10

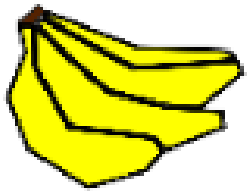
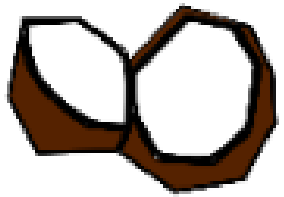
 = 10

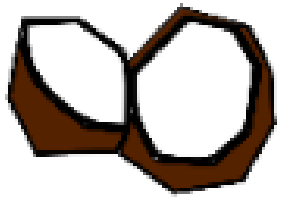
 +  +  = 18

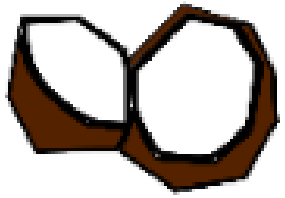
10 + 8  = 18

 = 1

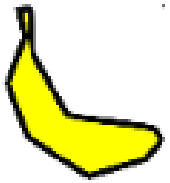
 = 1

 -  = 2

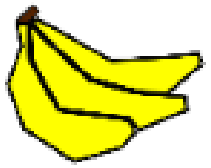
4 -  = 2

 = 2

 = 10

 = 1

 = 1  = 2

 +  +  = ??

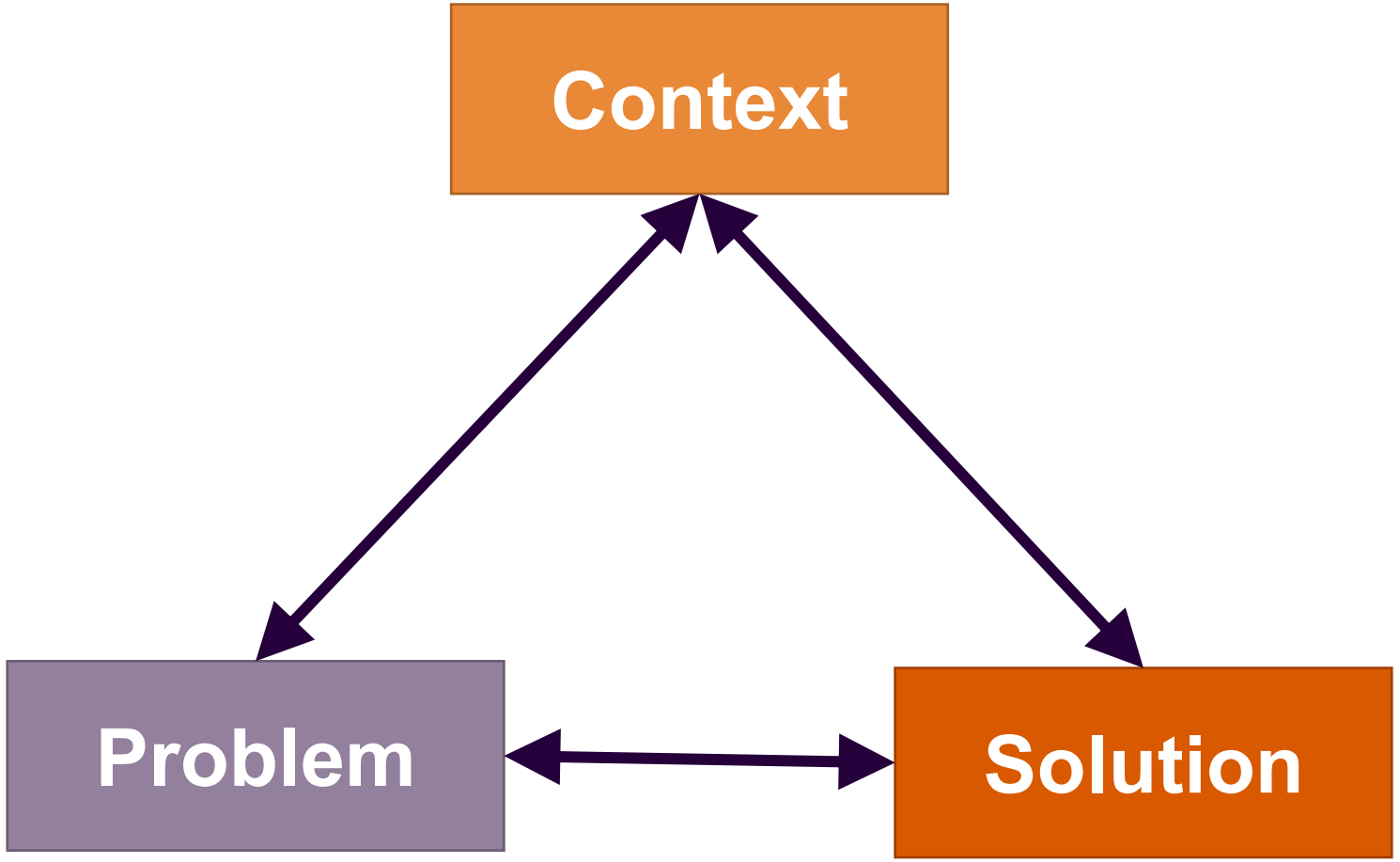
1 + 10 + 3 = 14

Considerations

- Economy
- Visibility
- Spacing
- Symmetry
- Emergence

Design Pattern

Design Patterns



Design Patterns

- Solves a problem
- Proven concept
- Solution isn’t obvious
- Describes a relationship
- Elegant

Design Patterns

- **Generative Patterns**
  - Describes a problem, a context, forces and pragmatic solution
- **Non-generative Patterns**
  - Describes a context and a problem, but no solution

Design Patterns

- **Abstraction and application**
  - Architectural patterns
  - Component patterns
  - Interface design patterns
  - WebApp patterns
  - Mobile patterns

Design Patterns

- **Creational patterns**
  - Creation, composition, representation of objects
- **Structural patterns**
  - How classes and objects are organised and integrated into a larger structure
- **Behavioural patterns**
  - Assignment of responsibility and communication between objects

Design Patterns

Examples of design patterns…

Design Patterns

- Visitor
- Observer
- Command (Action/Transaction)
- Adapter
- Proxy
- Façade
- Abstract Factory
- Object Pool

Design Patterns

- **Visitor** (Behavioral Pattern)
  - It adds new operations to existing objects without modifying their structure.
    - Visitor object is created
    - Element accepts the visitor
    - Visitor performs operation on the element

Design Patterns

- **Observer** (Behavioural Pattern)
  - It defines a one-to-many relationship between objects so that when one object (Subject) changes its state, all its dependent objects (Observers) are automatically notified.
    - Observers subscribe to subject
    - Subject state changes
    - Subject notifies all observers
    - Observers update automatically

Design Patterns

- **Command** (Behavioural Pattern)
  - Encapsulate a request as an object so it can be executed later.
  - Client creates a command object
  - Command is linked to a receiver
  - Invoker calls execute()
  - Receiver performs the action

Design Patterns

- **Adapter** (Structural Pattern)
  - It allows two incompatible interfaces to work together. It converts one interface into another that the client expects.
    - Client calls Adapter
    - Adapter translates request
    - Adaptee performs the action
    - Result is returned to client

Design Patterns

- **Proxy** (Structural Pattern)
  - It provides a placeholder (proxy object) that controls access to another object. A proxy acts as a substitute for a real object to control how it is accessed.
    - Client calls Proxy
    - Proxy checks conditions (security, caching, etc.)
    - Proxy forwards request to Real Object
    - Returns result

Design Patterns

- **Façade**
  - The Façade Pattern provides a simple interface to a complex system. It hides complexity and gives you an easy way to use a system.
    - Client calls the Façade
    - The Façade internally calls multiple subsystem methods
    - Returns simplified result to client

Design Patterns

- **Abstract Factory** (Creational Pattern)
  - It provides an interface for creating families of related objects without specifying their concrete classes. It lets you create multiple related objects together, without knowing their exact types.
    - Client selects a factory
    - Factory creates related objects
    - Client uses them via interfaces

Design Patterns

- **Object Pool** (Creational Pattern)
  - It manages a pool of reusable objects instead of creating and destroying them repeatedly. Instead of creating new objects every time, you reuse existing ones from a pool.
    - Pool creates a set of objects initially
    - Client requests an object, gets from pool
    - Client uses it
    - Client returns it to pool
    - Pool reuses it

Frameworks

Frameworks

- Patterns not sufficient
- Require implementation-specific skeletal infrastructure
  - Reusable mini-architecture that provides a generic structure and behaviour for a family of abstractions in a context
- Frameworks provide hooks and slots
- vs Design Patterns
  - Design patterns are more abstract
  - Design patterns are smaller architectural elements than frameworks
  - Design patterns are less specialised

Informatics 3A

Pattern-Based Design

UNIVERSITY OF JOHANNESBURG

Pareidolia

https://earthsky.org/human-world/seeing-things-that-arent-there

Pattern-based Software Design

Pattern-based Software Design

- Pattern-based Software Design in context
- Thinking in patterns
  - Understand big picture
  - Examine big picture, extracting patterns
  - Begin with big picture
  - Work inwards
  - Refine solution

Pattern-based Software Design Tasks

- Examine the requirements and develop a problem hierarchy
- Determine if a reliable patterns language has been developed for the domain
- Determine whether one or more architectural patterns are available
- Examine subsystem or component-level problems
  - Search for appropriate patterns
- Examine all problems

Pattern-Organising Table

	Database	Application	Implementation	Infrastructure
Data/Content				
Problem…		Pattern		Pattern
Problem…	Pattern		Pattern	
Architecture				
Problem…				Pattern
Problem…		Pattern		
Components				
Problem…				
Problem…				Pattern
User Interface				
Problem…		Pattern	Pattern	
Problem…		Pattern	Pattern	

Patterns in all Places

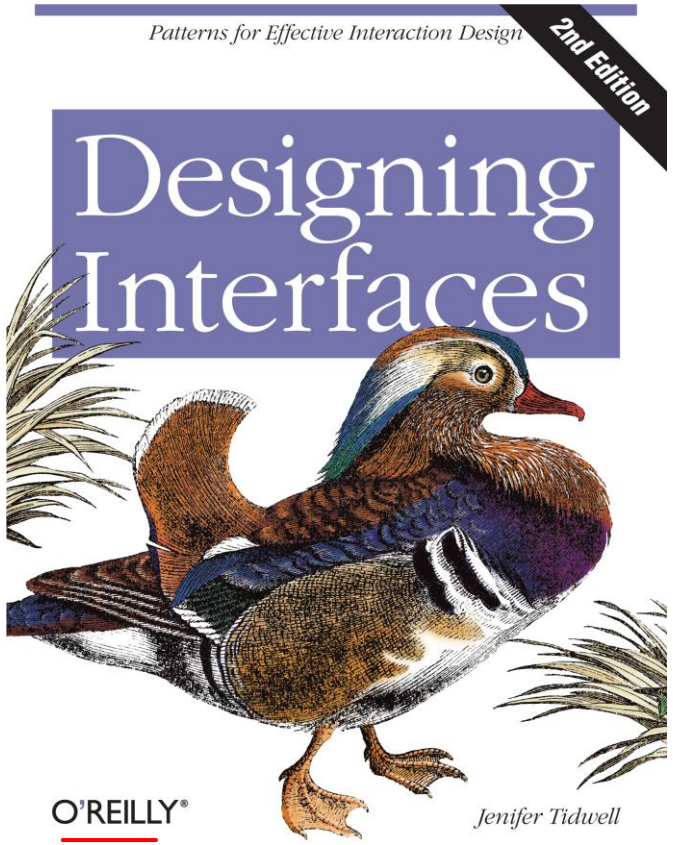
The Pattern You Are All Familiar With!

- **User Management**
  - Registration
  - Logging In
  - Forgot Password? (User password reset)
  - User Settings

Tidwell, J (2011). Designing Interfaces.

User Interface Design Patterns

1. Organising Content
2. Getting Around
3. Organising the Page
4. Lists of Things
5. Doing Things
6. Showing Complex Data
7. Getting Input from Users
8. Using Social Media
9. Going Mobile
10. Making pretty



Tidwell, J (2011). Designing Interfaces.

Mobility Design Patterns

- Check-in
- Sign-ups
- Maps
- Popovers
- Invitations

Anti-Patterns

- The Blob
- Spaghetti Code
- Cut-and-Paste Programming
- Lava Flow
- Walking through a Minefield
- Poltergeists
- Boat Anchor
- Golden Hammer
- Ambiguous Viewpoint
- Functional Decomposition